FSE 570 Capstone Project Proposal

Automotive Insurance Claims Risk Triage

Mriganko Chowdhury Aryan Gonsalves Ashish Raj Singh Deborah Sheryl Veluvalli Kshama Girish

February 2026

1 Introduction

Automotive insurers process a large volume of claims but have limited investigation capacity. Missed fraud increases losses, while over-reviewing legitimate claims increases cost and delays service. This project builds an end-to-end triage system that (1) predicts a fraud/risk score, (2) explains key drivers for each flagged claim, and (3) uses Retrieval-Augmented Generation (RAG) to produce a policy-grounded brief with citations to support routing decisions (approve, manual review, or SIU escalation).

2 Problem Statement

Given an auto insurance claim record, build a system that outputs a fraud/risk score, explains the main drivers (reason codes), and generates a policy-grounded brief using RAG with citations to support triage decisions (approve, manual review, or SIU). The system will be evaluated using imbalance-aware metrics such as PR-AUC and Precision@K.

3 Dataset

We use the *Vehicle Insurance Claim Fraud Detection* dataset from Kaggle.[1] It is a tabular CSV (`fraud_oracle.csv`) with ∼15,420 claim records and 33 attributes. The target label `FraudFound_P` is binary (1 = fraud, 0 = non-fraud).

Features cover claim timing/context, accident details, policy information, vehicle characteristics, and policyholder attributes. Since the data is class-imbalanced, we use stratified splits and imbalance-aware metrics. Preprocessing includes integrity checks, imputation (categorical → `Unknown`, numeric → median), and encoding (one-hot/ordinal) in a reproducible pipeline to prevent leakage.

4 Methodology

We build an end-to-end auto claims risk triage system that outputs (i) a fraud/risk score, (ii) per-claim explanations, and (iii) a policy-grounded brief via Retrieval-Augmented Generation (RAG).

- **Data Preparation:** Clean `fraud_oracle.csv` using a reproducible pipeline (integrity checks, imputation: categorical → `Unknown`, numeric → median, and encoding: one-hot for nominal; ordinal encoding where applicable). Use a stratified train/validation/test split to preserve class proportions.
- **Modeling and Triage:** Frame the task as binary classification with risk score $\hat{p} = P(\texttt{FraudFound\_P} = 1 \mid \mathbf{x})$. Train a baseline Logistic Regression and a stronger tabular model (LightGBM/XGBoost) with class weights. Route claims via a threshold or top-K policy into *Approve*, *Manual Review*, or *SIU*.
- **Explainability:** Use SHAP to compute per-claim feature contributions and convert the top drivers into human-readable reason codes to support analyst decisions.
- **RAG:** Curate claims-handling/fraud-guidelines documents, build a retrieval index, and use an LLM to generate a concise triage brief with citations to retrieved passages; if evidence is insufficient, explicitly report that support is not found in the documents.
- **Evaluation and Demo:** Evaluate with PR-AUC, Precision@K, and confusion-matrix error analysis; assess RAG using retrieval relevance and citation-grounded faithfulness. Demonstrate results in a

[1] `https://www.kaggle.com/datasets/shivamb/vehicle-claim-fraud-detection`

Streamlit app showing a risk-ranked review queue and claim details (score, reasons, cited brief).

5 Project Timeline

The project will be executed over a structured 12-week timeline, covering data preparation, modeling, explainability, RAG integration, and application development.

- **Weeks 1–2: Data Understanding and Preprocessing**
  - Explore dataset structure, feature distributions, and class imbalance.
  - Perform data integrity checks and identify missing values and anomalies.
  - Implement preprocessing pipeline including imputation, encoding, and stratified train/validation/test splits.
  - Establish reproducible pipeline using Python and scikit-learn.
- **Weeks 3–4: Baseline Modeling**
  - Train baseline Logistic Regression model.
  - Evaluate performance using PR-AUC, Precision, Recall, and confusion matrix.
  - Analyze model limitations and identify opportunities for improvement.
- **Weeks 5–6: Advanced Modeling**
  - Train gradient boosting model (LightGBM/XGBoost) with class imbalance handling.
  - Perform hyperparameter tuning using cross-validation.
  - Select best-performing model based on PR-AUC and Precision@K.
- **Weeks 7–8: Explainability and Reason Code Generation**
  - Compute SHAP values to explain individual predictions.
  - Identify top contributing features per claim.
  - Convert explanations into human-readable reason codes.
- **Weeks 9–10: RAG Integration**
  - Collect and preprocess insurance fraud policy and guideline documents.
  - Build retrieval index using embeddings and vector database.
  - Integrate LLM to generate claim triage briefs with citations.
  - Validate citation grounding and relevance.
- **Weeks 11–12: Application Development and Final Evaluation**
  - Develop Streamlit application for interactive claim triage visualization.
  - Display fraud risk scores, explanations, and generated briefs.
  - Perform final evaluation and error analysis.
  - Prepare documentation, report, and final demonstration.